

Prompt:

8/17/2026, 10:03:38 PM

lanjut Tahap R

Response:

8/17/2026, 10:03:39 PM

Tahap R — Robustness, Statistical Validation & Anti-Overfitting

Ya. Kalau Tahap Q adalah "**membangun dunia virtual tempat strategi diuji**", maka Tahap R adalah "**membuktikan bahwa hasil yang terlihat di dunia virtual itu bukan kebetulan.**"

Ini justru salah satu tahap paling penting untuk sistem kita.

Karena kita sekarang memiliki banyak komponen:

factor model → forecast → QUBO → QAOA/classical optimizer → execution → dynamic SL/TP

maka jumlah derajat kebebasan kita juga sangat besar.

Dan semakin banyak derajat kebebasan, semakin mudah kita secara tidak sengaja menemukan strategi yang **terlihat bagus hanya karena kita telah mencoba terlalu banyak hal.**

R.1. Masalah fundamental: backtest yang bagus belum tentu strategi yang bagus

Misalkan kita melakukan:

$$N = 10\,000$$

eksperimen.

Kita menemukan satu strategi:

$$Sharpe = 3.1.$$

Apakah:

$$H_1 : \text{strategy memiliki genuine edge}$$

benar?

Belum tentu.

Karena jika kita mencoba cukup banyak strategi, sebagian akan terlihat bagus **secara kebetulan**.

Ini disebut:

Multiple Testing / Data Snooping

R.2. Contoh sederhana

Misalkan seluruh strategi sebenarnya tidak mempunyai alpha:

$$E[R] = 0.$$

Kita mencoba 1.000 strategi.

Secara statistik, sangat mungkin beberapa memiliki:

$$Sharpe > 2,$$

bahkan:

$$Sharpe > 3.$$

Kemudian kita mengambil strategi terbaik.

Kita akan mengatakan:

"Lihat! Sharpe = 3.2!"

Padahal:

kita memilihnya karena ia menang secara kebetulan.

R.3. Inilah musuh utama kita

Ada beberapa bentuk:

Overfitting



Data Snooping

Selection Bias

Multiple Testing

Parameter Mining

Regime Overfitting

dan:

Backtest Overfitting.

R.4. Kita perlu membedakan dua konsep

In-sample performance

IS

adalah performa pada data yang digunakan untuk membangun model.

Out-of-sample performance

OOS

adalah performa pada data yang **tidak digunakan** untuk memilih model.

Tujuan kita:

$$Performance_{OOS} \approx Performance_{IS}$$

bukan:

$$Performance_{OOS} \rightarrow 0.$$

R.5. Tetapi kita bahkan tidak boleh hanya menggunakan satu OOS

Misalkan:

2021 – 2023

digunakan sebagai training.

Lalu:

2024

sebagai test.

Jika kita terus mengubah model berdasarkan hasil 2024, maka:

2024

secara perlahan berubah menjadi training data.

Karena itu kita membutuhkan:

Nested /Walk Forward Validation.

R.6. Walk-forward validation

Misalkan:

Train 1	Test 1
_____	_____
2021	2022 Q1
Train 2	Test 2
_____	_____
2021-Q1 2022	2022 Q2
Train 3	Test 3
_____	_____
2021-Q2 2022	2022 Q3

dan seterusnya.

Model selalu:

$$\boxed{Train \rightarrow Predict Future}$$

bukan:

$$Train \rightarrow Predict Past.$$

R.7. Purged walk-forward

Karena target kita dapat memiliki horizon:

$$H.$$

Misalnya:

$$H = 1h.$$

Jika sample training terakhir berada terlalu dekat dengan validation, targetnya bisa overlap.

Kita buang bagian tersebut.

Jika training berakhir:

$$t_{train},$$

validation dimulai:

$$t_{test},$$

maka:

$$t_{test} - t_{train} > H.$$

R.8. Embargo

Kita juga dapat menetapkan:

$$E > 0$$

sebagai embargo.

Sehingga:

$$[t_{train,end}, t_{test,start}]$$

memiliki gap.

Secara sederhana:

Train → Purging → Embargo → Test.

R.9. Mengapa ini sangat relevan untuk sistem kita?

Karena kita mempunyai feature seperti:

$$\mu_t,$$

$$\Sigma_t,$$

$$DXY_t,$$

$$Gold_t,$$

$$BTC_t,$$

$$ETH_t,$$

dan microstructure.

Beberapa di antaranya memiliki autocorrelation dan cross-correlation.

Jadi sample kita **tidak independent and identically distributed (IID)**.

R.10. Jangan menggunakan random train-test split

Misalnya:

```
train_test_split(X, y)
```

secara acak.

Ini berbahaya untuk time series.

Karena:

$$X_{2023}$$

dapat berada di training sementara:

$$X_{2022}$$

berada di test.

Model secara tidak langsung melihat struktur masa depan.

Untuk sistem kita:

Temporal Split Only.

R.11. Distribution shift

Pasar tidak stationary.

Secara ideal kita ingin:

$$P_{train}(X, Y) = P_{test}(X, Y).$$

Tetapi kenyataannya:

$$P_{train}(X, Y) \neq P_{test}(X, Y).$$

Ini disebut:

Distribution Shift.

R.12. Regime

Misalkan:

$$Z_t \in \{Bull, Bear, Sideways, HighVol, LowVol\}.$$

Maka:

$$P(R_t|Z_t)$$

dapat berbeda drastis.

Karena itu performa harus dianalisis:

$$E[R|Z = Bull],$$

$$E[R|Z = Bear],$$

dst.

R.13. Strategi yang hanya bekerja di satu regime

Misalnya:

$$Sharpe_{Bull} = 3.5$$

tetapi:

$$Sharpe_{Bear} = -2.0.$$

Strategi tersebut bukan otomatis buruk.

Tetapi kita perlu memahami:

conditional edge.

Mungkin strategi memang dirancang untuk:

Bullregime.

R.14. Regime detector

Kita dapat mendefinisikan:

$$Z_t = f(F_t)$$

dengan faktor:

$$F_t = [\sigma_t, DXY_t, Gold_t, VIX_t, BTC_t, SPX_t, \dots].$$

Kemudian strategi:

$$\pi_t = \pi(S_t, Z_t).$$

R.15. Tetapi jangan overfit regime

Ini bahaya.

Misalnya kita menemukan:

"Strategi bekerja bagus jika VIX antara 17.23 dan 21.87."

Kemudian:

"DXY harus naik 0.17%."

Kemudian:

"Gold harus turun 0.08%."

Kita telah membuat filter yang sangat spesifik.

Itu kemungkinan besar:

overfitting.

R.16. Parameter stability

Misalkan parameter strategi:

$$\theta = (\lambda, \gamma, \beta, p).$$

Kita tidak ingin hanya:

$$\theta^*$$



yang menghasilkan performa bagus.

Kita ingin neighborhood:

$$N(\theta^*)$$

juga relatif bagus.

Secara visual:

Performance surface should be smooth.

R.17. Contoh

Buruk:

$$Sharpe(1.01) = 0.8$$

$$Sharpe(1.02) = 0.9$$

$$Sharpe(1.03) = 3.8$$

$$Sharpe(1.04) = 0.7.$$

Ini mencurigakan.

Lebih sehat:

$$2.0, 2.1, 2.2, 2.1, 2.0.$$

R.18. Robust optimum

Idealnya:

$$\boxed{\theta^*}$$

berada di area yang relatif datar.

Bukan puncak tajam.

Secara matematis:

$$\nabla_{\theta} J(\theta^*) \approx 0$$

dan:

$$\nabla_{\theta}^2 J$$

tidak menghasilkan permukaan yang terlalu tajam.

Ini sebenarnya sangat mirip dengan konsep **robust optimization**.

R.19. Robust optimization

Daripada:

$$\max_{\theta} E[J(\theta)],$$

kita dapat mempertimbangkan:

$$\max_{\theta} \min_{P \in \mathcal{U}} E_P[J(\theta)]$$

dengan:

$$\mathcal{U}$$

adalah uncertainty set distribusi pasar.

Artinya kita tidak mencari strategi yang terbaik pada satu estimasi distribusi.

Kita mencari strategi yang:

tetap cukup baik ketika asumsi kita salah.

R.20. Ini sangat cocok dengan proyek kita

Karena:

$$\hat{\mu}$$

dan:

$$\hat{\Sigma}$$

tidak pernah diketahui secara sempurna.

Misalnya:

$$\hat{\mu} = \mu + \epsilon_{\mu}.$$

$$\hat{\Sigma} = \Sigma + \epsilon_{\Sigma}.$$

Maka optimizer harus robust terhadap:

$$(\epsilon_{\mu}, \epsilon_{\Sigma}).$$

R.21. Stress test terhadap covariance

Misalkan:

$$\Sigma.$$

Kita dapat membuat:

$$\Sigma' = \Sigma + \Delta\Sigma.$$

Dengan:

$$\|\Delta\Sigma\| \leq \epsilon.$$

Kemudian jalankan optimizer.

Jika strategi langsung rusak:

model terlalu sensitif terhadap estimation error.

R.22. Stress test terhadap expected return

Demikian juga:

$$\mu' = \mu + \Delta\mu.$$

Kemudian:

$$\|\Delta\mu\| \leq \epsilon.$$

Kita lihat apakah portfolio/order decision berubah drastis.

R.23. Sensitivity terhadap transaction cost

Misalkan base:

$$c = 0.05\%.$$

Kemudian:

$$c \in \{0.025\%, 0.05\%, 0.10\%, 0.20\%, 0.50\%\}.$$

Kita hitung:

$$PnL(c).$$

Jika:

$$PnL(0.05\%) > 0$$

tetapi:

$$PnL(0.10\%) < 0,$$

maka edge kita sangat tipis.

R.24. Slippage stress

Demikian juga:

$$s \in \{0.5s_0, s_0, 2s_0, 3s_0\}.$$

Kita ingin:

$$PnL(s)$$

tetap masuk akal.

R.25. Liquidity stress

Kita dapat menurunkan:

$$Depth$$

secara artifisial.

Misalnya:

$$D' = 0.5D.$$

Kemudian:

$$P_{impact}$$

akan meningkat.

Jika strategi langsung collapse:

$$\boxed{strategy\ liquidity - sensitive.}$$

R.26. Monte Carlo return bootstrap

Misalkan kita memiliki trade return:

$$r_1, \dots, r_N.$$

Kita dapat melakukan bootstrap:

$$r_1^*, \dots, r_N^*$$

dengan sampling ulang.

Kemudian:

$$Equity_t^*.$$

Ulangi:

$$B = 10\,000.$$

Kita memperoleh distribusi:

$$MDD^{(1)}, \dots, MDD^{(B)}.$$

R.27. Tetapi IID bootstrap tidak selalu cocok

Karena return financial time series mempunyai:

- autocorrelation,
- volatility clustering,
- regime persistence.

Maka lebih baik:

Block Bootstrap.

R.28. Block bootstrap

Kita membagi return:

$$R_1, \dots, R_T$$

menjadi blok:

$$B_1, B_2, \dots, B_K.$$

Kemudian sampling blok.

Dengan demikian struktur lokal tetap dipertahankan.

R.29. Stationary bootstrap

Alternatif yang lebih fleksibel:

$$L \sim \text{Geometric}(p)$$

untuk menentukan panjang blok.

Kemudian blok dimulai secara random.

Ini menghasilkan:

Stationary Bootstrap.

R.30. Yang kita cari bukan hanya confidence interval PnL

Kita ingin:

$$P(\text{Sharpe} > 0),$$

$$P(MDD < MDD_{max}),$$

$$P(\text{Capital} > 0),$$

dan:

$$P(\text{ruin}).$$

R.31. Probability of ruin

Misalkan capital:

$$C_0.$$

Threshold:

$$C_{min}.$$

Kita definisikan:

$$P_{ruin} = P(\min_t C_t < C_{min}).$$

Ini jauh lebih berguna daripada sekadar melihat maximum drawdown historis.

R.32. Deflated Sharpe Ratio

Sekarang kita masuk ke alat yang sangat penting.

Sharpe ratio observasi:

$$SR_{obs}.$$

Jika kita mencoba banyak strategi, maximum Sharpe yang kita lihat akan cenderung meningkat walaupun tidak ada skill.

Kita perlu memperhitungkan:

selection bias.

Inilah motivasi:

Deflated Sharpe Ratio

yang secara konseptual menyesuaikan Sharpe berdasarkan:

- number of trials,
- non-normality,
- skewness,
- kurtosis,
- sample length,
- correlation antar strategi.

R.33. Mengapa sangat relevan bagi kita?

Karena kita mungkin mencoba:

$$QUBO_1, QUBO_2, \dots, QUBO_N.$$

Kemudian:

$$\theta_1, \dots, \theta_M.$$

Kemudian:

$$SL_1, \dots, SL_K.$$

Jumlah kombinasi bisa menjadi:

$$N \times M \times K.$$

Jika kita kemudian mengambil yang terbaik:

selection bias

hampir pasti menjadi masalah.

R.34. Effective number of trials

Tidak selalu:

$$N_{trials} = N.$$

Jika strategi sangat berkorelasi:

$$\rho_{ij} \approx 1,$$

maka informasi yang diperoleh dari eksperimen tersebut tidak independen.

Kita dapat mempertimbangkan:

$$N_{eff}.$$

Ini lebih masuk akal daripada sekadar menghitung jumlah file eksperimen.

R.35. Probability of Backtest Overfitting

Salah satu konsep penting berikutnya:

$$PBO$$

atau:

Probability of Backtest Overfitting.

Ide dasarnya:

Apakah strategi yang kita pilih sebagai pemenang in-sample juga cenderung menjadi pemenang out-of-sample?

Jika tidak:

PBO

tinggi.

R.36. Combinatorial Purged Cross Validation

Untuk time series, kita dapat menggunakan:

CPCV

yaitu:

Combinatorial Purged Cross Validation.

Data dibagi menjadi beberapa kelompok temporal.

Kemudian berbagai kombinasi train/test dibuat sambil menerapkan:

purging

dan:

embargo.

R.37. Mengapa ini lebih kuat?

Misalkan kita memiliki:

$N = 6$

fold:

$F_1, \dots, F_6.$

Kita tidak hanya mempunyai satu test period.

Kita dapat membuat banyak jalur:

$$F_1, F_3, F_5$$

sebagai test pada kombinasi tertentu, misalnya dengan tetap menjaga urutan temporal dan aturan purging.

Akhirnya kita memperoleh distribusi performa OOS.

R.38. White's Reality Check

Sekarang masalah:

"Saya mencoba banyak strategi. Apakah strategi terbaik saya benar-benar memiliki predictive ability?"

Kita dapat menguji:

$$H_0 : E[R_i] \leq 0$$

untuk seluruh strategi i .

Alternative:

$$H_1 : \exists i : E[R_i] > 0.$$

White's Reality Check dirancang untuk memperhitungkan data snooping dalam evaluasi banyak model.

R.39. Hansen's SPA

Alternatif yang juga sangat relevan:

Superior Predictive Ability

atau SPA test.

Pertanyaannya:

Apakah strategi terbaik benar-benar mengalahkan benchmark setelah memperhitungkan data snooping?

Ini lebih dekat dengan kebutuhan kita daripada hanya:

$$Sharpe > 1.$$

R.40. Multiple hypothesis testing

Misalkan kita menguji:

$$H_1, H_2, \dots, H_m.$$

Jika masing-masing menggunakan:

$$\alpha = 0.05,$$

probabilitas setidaknya satu false positive meningkat seiring m .

Jika test independen:

$$P(\text{at least one false positive}) = 1 - (1 - \alpha)^m.$$

Untuk:

$$m = 100,$$

maka:

$$1 - 0.95^{100} \approx 99.4\%.$$

Itulah sebabnya:

$$p < 0.05$$

sendirian tidak cukup.

R.41. False Discovery Rate

Kita dapat menggunakan:

FDR

misalnya prosedur:

Benjamini – Hochberg.

Ini berguna ketika kita mengevaluasi banyak faktor atau banyak hipotesis alpha.

R.42. Tetapi ada satu masalah penting

Dalam trading, kita sering melakukan:

"Coba parameter."

Kemudian:

"Tidak bagus."

Coba lagi.

Kemudian:

"Masih tidak bagus."

Coba lagi.

Sampai:

"Wow, Sharpe 2.8!"

Secara statistik kita telah melakukan:

implicit multiple testing.

Walaupun kita tidak menulis 1.000 eksperimen secara eksplisit.

R.43. Karena itu experiment registry wajib

Setiap eksperimen harus dicatat:

ExperimentID

DataVersion

FeatureVersion

ModelVersion

ParameterSet

Seed

Timestamp.

R.44. Jangan menghapus eksperimen buruk

Ini penting.

Jika:

$E_1 : \text{Sharpe} = -0.3$

$E_2 : \text{Sharpe} = 0.2$

$E_3 : \text{Sharpe} = 2.1$

kita tidak boleh hanya menyimpan:

$E_3.$

Karena jumlah percobaan:

$N_{\text{trials}} = 3.$

bukan:

1.

R.45. Ini juga berlaku untuk QAOA

Misalnya kita menguji:

$$p \in \{1, 2, 3, 4, 5\}$$

dan:

$$\beta, \gamma$$

dengan banyak kombinasi.

Jika kita memilih kombinasi yang menghasilkan PnL terbesar:

$$(\beta^*, \gamma^*, p^*),$$

maka pencarian tersebut sendiri adalah:

model selection.

Harus dihitung sebagai bagian dari trial history.

R.46. Bahkan ansatz adalah hyperparameter

Misalnya:

EfficientSU2

vs.

UCC

vs.

hardware – efficient.

Jika kita mencoba semuanya lalu memilih yang terbaik, itu juga:

multiple testing.

R.47. Dan optimizer

Misalnya:

SPSA,

Adam,

COBYLA,

Nelder – Mead.

Pemilihan optimizer berdasarkan hasil test juga merupakan:

modelselection.

R.48. Kita perlu membedakan tiga dataset

Ini sangat penting.

Training

digunakan untuk:

fit.

Validation

digunakan untuk:

model selection.

T_{test}

digunakan hanya untuk:

final evaluation.

R.49. Test set adalah "vault"

Idealnya:

Test set tidak disentuh sampai model dibekukan.

Jika kita melihat hasil test kemudian mengubah model:

$T_{test} \rightarrow T_{training}$.

R.50. Final evaluation

Setelah model benar-benar dibekukan:

θ_{final}

baru kita jalankan:

$T_{test_{final}}$.

Dan tidak boleh melakukan tuning lagi berdasarkan hasil tersebut.

R.51. Statistical significance

Misalkan rata-rata trade return:

$\bar{r}$.

Sample standard deviation:

s .



Standard error sederhana:

$$SE(\bar{r}) = \frac{s}{\sqrt{N}}.$$

Test statistic:

$$t = \frac{\bar{r}}{SE(\bar{r})}.$$

Tetapi financial returns sering:

- non-normal,
- autocorrelated,
- heteroskedastic.

Jadi asumsi sederhana ini harus diperlakukan hati-hati.

R.52. HAC / Newey-West

Jika terdapat autocorrelation, variance estimator dapat diperbaiki menggunakan:

Heteroskedasticity and Autocorrelation Consistent

atau HAC.

Ini memungkinkan standard error lebih realistis.

R.53. Sharpe juga memiliki masalah

Standard Sharpe:

$$SR = \frac{\bar{r}}{s}.$$

Jika return non-normal, Sharpe tidak sepenuhnya menggambarkan risk.

Kita harus memperhatikan:

Skewness

dan:

Kurtosis.

R.54. Tail risk

Kita dapat mengukur:

$$VaR_{\alpha}.$$

Misalnya:

$$VaR_{95\%}.$$

Tetapi saya lebih menyukai:

$$\boxed{CVaR}$$

atau Expected Shortfall:

$$ES_{\alpha} = E[L|L \geq VaR_{\alpha}].$$

R.55. Mengapa?

VaR menjawab:

"Pada quantile 95%, seberapa besar loss?"

Sedangkan ES menjawab:

"Jika kita sudah masuk 5% kejadian terburuk, rata-rata loss-nya berapa?"

Untuk tail-heavy financial distributions, ini sangat penting.

R.56. Drawdown distribution

Jangan hanya:

MDD .

Kita juga dapat melihat:

$Duration_{DD}$,

$RecoveryTime$,

$AverageDD$,

$ExpectedDD$.

Karena:

$MDD = 10\%$

selama 2 hari berbeda dengan:

$MDD = 10\%$

selama 8 bulan.

R.57. Recovery time

Misalkan equity mencapai peak:

E_{peak} .

Kemudian turun.

Recovery time:

$$T_{recover} = t_{new\ peak} - t_{drawdown}.$$

Ini sangat penting untuk strategi short-term yang mungkin memiliki banyak drawdown kecil.

R.58. Capacity analysis

Ini satu hal yang sangat sering dilupakan.

Jika strategy menghasilkan:

\$100

profit dengan:

\$1,000

capital, bagus.

Tetapi apakah bisa:

\$10,000,000?

Belum tentu.

Karena:

$MarketImpact(q)$

meningkat dengan:

q .

R.59. Impact model

Secara sederhana:

$$Impact(q) = \eta \sigma \left(\frac{q}{V} \right)^\alpha$$

dengan:

- q : order size,
 - V : market volume,
 - σ : volatility,
 - η, α : parameter.
-

R.60. Capacity

Kita dapat mencari:

$$q^*$$

sehingga:

$$ExpectedAlpha(q^*) = TransactionCost(q^*).$$

Di atas:

$$q > q^*,$$

strategy tidak lagi economic.

Jadi:

$$\boxed{Capacity}$$

adalah bagian dari robustness.

R.61. Latency stress

Untuk short-term trading, kita juga harus menguji:

$$\tau$$

sebagai latency.

Misalnya:

$$\tau \in \{0, 10, 50, 100, 500\} \text{ ms.}$$

Kemudian:

$$PnL(\tau).$$

Jika edge hilang pada:

$$\tau = 50ms,$$

strategi kita mungkin membutuhkan infrastructure yang sangat cepat.

R.62. Execution degradation curve

Kita dapat mendefinisikan:

$$D(\tau) = P nL(0) - P nL(\tau).$$

Ini memberikan gambaran:

seberapa cepat alpha kita decay.

R.63. Alpha half-life

Misalkan signal:

$$S_t.$$

Autocorrelation:

$$\rho(k).$$

Kita dapat mencari:

$$k_{1/2}$$

ketika:

$$\rho(k_{1/2}) \approx 0.5.$$

Itulah pendekatan sederhana untuk:

$$\textit{Alpha Half - Life}.$$

Jika half-life sangat pendek, execution menjadi kritikal.

R.64. Ini menghubungkan Tahap R dengan Tahap Q

Tahap Q:

execution simulation.

Tahap R:

berapa sensitif hasil terhadap execution assumptions?

Jadi:

$$Q \rightarrow R$$

secara alami.

R.65. Robustness terhadap universe

Misalnya awalnya:

$$Universe = \{BTC, ETH, SOL, \dots\}.$$

Kita harus menguji:

$$Universe_1, Universe_2, Universe_3.$$

Jika strategi hanya bekerja pada satu subset yang dipilih setelah melihat hasil:

$$selectionbias.$$

R.66. Cross-market validation

Karena tujuan kita bukan hanya crypto, kita dapat menguji:

Crypto

US Equities

European Equities

Asian Equities

FX

jika data dan execution model mendukung.

R.67. Tetapi hati-hati

Kita tidak boleh mengatakan:

"Model bekerja di BTC, maka pasti bekerja di saham."

Justru kita harus menguji apakah struktur matematisnya transferable.

Misalnya:

factor model

mungkin transferable.

Tetapi:

microstructure

jelas berbeda.

R.68. Hierarchical robustness

Saya ingin robustness kita dibagi menjadi empat tingkat:

Level 1 — Statistical

Significance

Level 2 — Temporal

OOS / Walk-forward

Level 3 — Market

Regime / Asset / Geography

Level 4 — Operational

Fees / Slippage / Latency / Liquidity.

Jika model melewati keempatnya, tingkat kepercayaan jauh lebih tinggi.

R.69. Robustness score

Kita bahkan dapat membangun dashboard internal:

$$R_{score} = w_1 R_{stat} + w_2 R_{temporal} + w_3 R_{market} + w_4 R_{operational}.$$

Tetapi **jangan menggunakan skor ini sebagai pengganti statistical test.**

Ini hanya alat ringkasan.

R.70. Stress matrix

Saya menyarankan kita membuat:

Stress	Parameter
Cost	1 – 5×
Slippage	1 – 5×
Latency	0 – 500ms+
Liquidity	0.1 – 1.0×
Volatility	0.5 – 3×
Factor error	±ε
Covariance error	±ΔΣ
Fill probability	±ε

Kemudian:

$$Performance = f(Cost, Slippage, Latency, Liquidity, \dots).$$

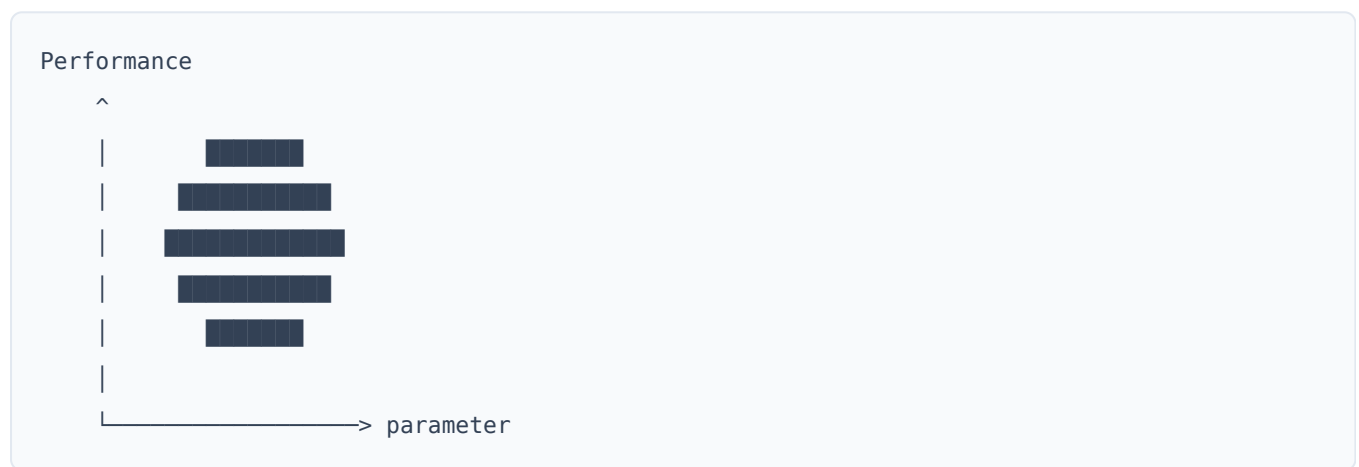
R.71. Yang kita cari bukan "profit maksimum"

Kita ingin:

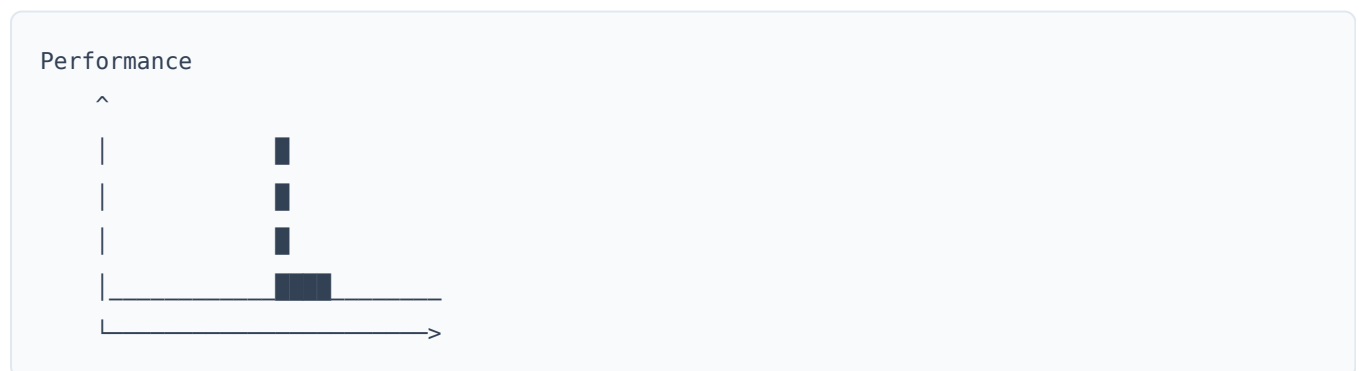
Stable Positive Region

dalam ruang parameter.

Secara visual:



Bukan:



R.72. Dari sini kita bisa mendefinisikan "robust alpha"

Saya mengusulkan definisi operasional:

Robust Alpha = Positive OOS Edge + Statistical Evidence + Parameter Stability + Regime Robustness + Execution

Ini jauh lebih kuat daripada:

$$\text{Sharpe} > 2.$$

R.73. Dan sekarang bagian yang sangat penting untuk quantum

Kita memiliki tiga hipotesis berbeda.

H1 — Optimization hypothesis

Apakah QAOA mampu menyelesaikan formulasi optimasi dengan kualitas yang memadai?

H2 — Trading hypothesis

Apakah formulasi optimasi tersebut menghasilkan keputusan trading yang memiliki edge?

H3 — Quantum advantage hypothesis

Apakah QAOA memberikan nilai ekonomi atau komputasional yang tidak diperoleh classical optimizer?

Ketiganya:

bukan hipotesis yang sama.

R.74. Ini mencegah kesalahan besar

Misalkan:

QAOA

menghasilkan solusi bagus.

Itu membuktikan:

H_1 .

Belum membuktikan:

$$H_2.$$

Dan bahkan jika:

$$H_2$$

benar, belum membuktikan:

$$H_3.$$

R.75. Quantum comparison matrix

Kita akhirnya akan membandingkan:

Model	Solution Quality	Runtime	PnL	Sharpe	MDD	Cost
Exact	100%	—	—	—	—	—
Classical	—	—	—	—	—	—
QAOA	—	—	—	—	—	—

Dan:

$$\boxed{\textit{Economic Value}}$$

harus menjadi salah satu kolom utama.

R.76. "Quantum advantage" yang realistis

Saya tidak ingin kita mendefinisikan:

$$\textit{Quantum Advantage} = PnL_{\text{QAOA}} > PnL_{\text{classical}}.$$

Itu definisi yang buruk.

Karena optimizer yang berbeda bisa menghasilkan keputusan berbeda akibat stochasticity.

Lebih masuk akal:

$$\text{Quantum Advantage} = f(\text{solution quality, runtime, scalability, economic value}).$$

R.77. Scaling test

Misalnya jumlah aset:

$$N = 4, 8, 12, 16, 20, \dots$$

Kita ukur:

$$T_{\text{classical}}(N)$$

dan:

$$T_{\text{QAOA}}(N).$$

Juga:

$$\text{Gap}(N).$$

Maka kita memperoleh:

$$\text{Scaling Curve.}$$

R.78. Ini mungkin menjadi kontribusi ilmiah yang lebih kuat

Daripada:

"Saya menggunakan QAOA untuk trading dan menghasilkan profit."

Lebih ilmiah jika:

"Kami mengevaluasi hubungan antara approximation quality, computational cost, dan economic degradation pada dynamic financial optimization problem di bawah realistic transaction-cost and execution constraints."

Itu sudah menjadi penelitian yang jauh lebih serius.

R.79. Statistical validation pipeline final

Saya ingin Tahap R kita akhirnya berbentuk:

RAW DATA



TEMPORAL SPLIT



PURGED WALK-FORWARD



MODEL TRAINING



VALIDATION



MODEL SELECTION



MODEL FREEZE



FINAL OOS TEST



BOOTSTRAP



CONFIDENCE
INTERVAL



STRESS TEST



ROBUSTNESS
ANALYSIS



MULTIPLE TESTING



DSR / PBO / SPA



FINAL EVIDENCE

R.80. Decision gate

Setelah Tahap R, saya ingin sistem kita mempunyai **gate**.

Misalnya:

$$G_{stat}$$

untuk statistical evidence,

$$G_{OOS}$$

untuk out-of-sample,

$$G_{robust}$$

untuk robustness,

dan:

$$G_{execution}$$

untuk execution.

Kemudian:

$$\boxed{Deploy = G_{stat} \wedge G_{OOS} \wedge G_{robust} \wedge G_{execution}.}$$

Kalau salah satu gagal:

$$\boxed{NO\ LIVE\ DEPLOYMENT.}$$

R.81. Saya bahkan menyarankan tiga status

RED

$$\boxed{Insufficient\ Evidence}$$

Strategi belum boleh dipakai.

YELLOW

Promising but Uncertain

Lanjutkan penelitian/paper trading.

GREEN

Robust OOS Evidence

Baru boleh masuk tahap paper/live dengan risk limit yang sangat kecil.

R.82. Dan ini sangat cocok dengan filosofi \$1 risk kita

Risk budget tetap:

$$R_{trade} \leq \$1.$$

Tetapi sekarang kita menambahkan:

$$R_{model}$$

dan:

$$R_{execution}$$

Jadi:

$$TotalRisk \ni Risk_{trade}$$

saja.

Kita memiliki:

$$Total\ System\ Risk = Market + Model + Execution + Liquidity + Operational.$$

R.83. Model risk

Misalnya:

$$\hat{\mu}$$

salah.

Maka:

$$QUBO$$

dapat menghasilkan keputusan yang salah.

Jadi:

$$\boxed{Model\ uncertainty \rightarrow Decision\ uncertainty.}$$

Ini harus masuk dalam stress testing.

R.84. Decision uncertainty

Jika dua solusi memiliki objective yang hampir sama:

$$E(x_1) \approx E(x_2),$$

tetapi keputusan trading sangat berbeda:

$$x_1 \not\equiv x_2,$$

maka sistem tidak boleh terlalu percaya diri.

Kita dapat mendefinisikan:

$$\Delta E = |E(x_1) - E(x_2)|.$$

Jika:

$$\Delta E \ll \epsilon,$$

kita berada di daerah:

R.85. Ini membuka jalan untuk tahap berikutnya

Karena jika beberapa solusi hampir sama:

$$x_1, x_2, \dots, x_k,$$

kita tidak harus selalu memilih:

$$x^*.$$

Kita bisa membangun:

solution ensemble.

atau bahkan:

$$P(x)$$

sebagai distribusi keputusan.

Ini akan sangat menarik jika dikombinasikan dengan output probabilistik QAOA.

R.86. Dan di sinilah proyek kita mulai menjadi jauh lebih dalam

Kita awalnya ingin:

Trading Bot.

Sekarang arsitektur kita telah berkembang menjadi:

Statistical Decision System under Uncertainty

dengan:

$$\underbrace{\text{Market Uncertainty}}_{\epsilon_M}$$

Model Uncertainty
 ϵ_θ

Optimization Uncertainty
 ϵ_Q

Execution Uncertainty .
 ϵ_E

Dan keputusan:

$$A_t$$

harus tetap robust terhadap keempatnya.

R.87. Target akhir Tahap R

Pada akhir Tahap R, kita seharusnya tidak lagi bertanya:

"Apakah backtest menghasilkan profit?"

Pertanyaan kita berubah menjadi:

"Seberapa kuat bukti bahwa expected economic edge tetap positif ketika kita memperhitungkan temporal dependence, multiple testing, model uncertainty, regime changes, transaction costs, execution uncertainty, dan optimization error?"

Secara matematis:

$$P(E[R_{OOS}] > 0 \mid D, M, E)$$

harus cukup tinggi, dengan:

- D: data,
- M: model assumptions,
- E: execution assumptions.

Dan bahkan itu belum berarti kita "membuktikan profit". Yang kita dapatkan adalah **evidence yang lebih kuat atau lebih lemah** terhadap adanya edge.

Posisi kita setelah Tahap R

Kita sekarang memiliki:

$A - P$: Mathematical Trading System

Q : Event-Driven Market Simulator

R : Statistical Validation & Robustness

Sehingga pipeline besarnya menjadi:

Market Data → State Estimation → Alpha → Optimization → Execution → Simulation → Statistical Validation

◀  ▶

Dan menurut saya, **Tahap S berikutnya sebaiknya bukan langsung masuk live trading**. Kita perlu membangun **S — Portfolio/Strategy Ensemble & Capital Allocation**, yaitu bagaimana beberapa strategi/optimizer/factor model yang masing-masing telah lolos validasi R digabungkan secara matematis menjadi satu *meta-portfolio*, termasuk korelasi antar-strategi, risk budgeting, Kelly/fractional Kelly, hierarchical risk parity, regime-dependent allocation, dan—yang sangat cocok dengan arah penelitian kita—**formulasi capital allocation tersebut sebagai QUBO/Ising problem**.